

Concurrency in ROS 2 (Python)

Executors, callback groups, the GIL, processes, and the other things that stall a control loop

2026-09-27

Table of contents

1	Why this document exists	1
2	The pieces of a ROS 2 node	2
3	Executors	2
3.1	SingleThreadedExecutor (rclpy and rclcpp)	2
3.2	MultiThreadedExecutor (rclpy and rclcpp)	3
3.3	Callback groups: the part that actually allows parallelism	3
3.4	StaticSingleThreadedExecutor (rclcpp only)	4
3.5	EventsExecutor (rclcpp, and rclpy.experimental since Jazzy)	4
4	Python-specific: the GIL	4
4.1	Thread safety with a GIL	6
5	Threads vs. processes	7
6	Stop-the-world pauses that aren't your code	8
6.1	Garbage collection	8
6.2	Just-in-time compilation in flight	8
6.3	Printing	9
7	Checklist for a Python ROS 2 control node	9

1 Why this document exists

The `px4_rta_mm_gpr` node has several periodic jobs: a 100 Hz control law, a 10 Hz offboard heartbeat, a 10 Hz wind estimator, and a reachability *rollout* that takes about 165 ms on this machine. The original code gave each job its own timer, and it was natural to assume that separate timers run independently, so a slow rollout couldn't delay the controller.

They don't. `rclpy.spin(node)` runs every callback of the node, one after another, on a single thread. Flying the original node in SITL on this machine gave these numbers:

	original node
control rate during the RTA phase	12 Hz (target 100 Hz)
median / worst control period	26.6 ms / 1353 ms
time spent inside rollouts during the RTA phase	~14 s of 20 s
outcome	drifted 57 m sideways and hit the ground

This document explains the machinery behind that result, and each tool used to fix it, from the ground up. The companion document (`02_rta_node_changes`) describes the changes to this node and the measured results.

2 The pieces of a ROS 2 node

Callbacks. Nothing in a ROS 2 node runs “on its own”. Every piece of work is a *callback* that some event triggers:

- a **timer** callback, when its period elapses (`create_timer(0.01, f)`);
- a **subscription** callback, when a message arrives;
- service, client, action and guard-condition callbacks, which follow the same idea.

Creating a timer does not create a thread. It only registers the fact that `f` should be called every 10 ms by *whoever is running the node*.

The executor is that whoever. It is a loop that:

1. waits on a *wait set* (timers, subscriptions, ...) until at least one entity is ready;
2. picks a ready entity whose callback is allowed to run (see callback groups below);
3. calls the callback, then goes back to 1.

`rclpy.spin(node)` is shorthand for “make a `SingleThreadedExecutor`, add `node` to it, and call `spin()`”. **The executor, not the node, decides whether callbacks can overlap in time.**

Timers do not queue missed ticks. If a 100 Hz timer can’t run for 165 ms because the executor is busy, it doesn’t run 16 times in a burst afterwards. It runs *once*, as soon as it can, and then resumes its schedule. The missed ticks are simply gone, which is why the original node measured an average of 12 Hz rather than 100 Hz with a backlog.

3 Executors

3.1 `SingleThreadedExecutor` (`rclpy` and `rclcpp`)

One thread runs every callback in sequence. It is simple and needs no locks, because nothing is ever concurrent. The cost is that **the latency of any callback is bounded below by the longest callback that can be scheduled ahead of it**. With a 165 ms rollout in the node, every other callback, including the 100 Hz controller, can be delayed by up to 165 ms (panel (a) of Figure 1).

3.2 MultiThreadedExecutor (rcipy and rclcpp)

A pool of worker threads. The spinning thread still waits on the wait set, but it *hands* each ready callback to a free worker, so several callbacks can run at once. In rcipy it is a `concurrent.futures.ThreadPoolExecutor` underneath. Its default size is the number of CPUs the process may use (`os.sched_getaffinity`).

On its own, switching to `MultiThreadedExecutor` changes **nothing** for this node, for the reason in the next section.

3.3 Callback groups: the part that actually allows parallelism

Every callback belongs to a *callback group*. A group states which of its own callbacks may run concurrently:

- **MutuallyExclusiveCallbackGroup**: at most one callback from the group runs at any time.
- **ReentrantCallbackGroup**: any number may run at once, including the *same* callback overlapping with itself.

Callbacks in *different* groups may always run in parallel on a multithreaded executor.

The catch: **if you don't pass `callback_group=`, every callback goes into the node's default group, which is one `MutuallyExclusiveCallbackGroup`**. A `MultiThreadedExecutor` running a node whose callbacks all share one mutually exclusive group behaves exactly like a single-threaded one. The executor has threads, but the group allows only one callback at a time. Parallelism needs *both*: a multithreaded executor *and* callbacks split across groups.

The refactored node uses five mutually exclusive groups:

```
self.px4_io_group      = MutuallyExclusiveCallbackGroup() # heartbeat,
    ↪ vehicle_status, RC
self.state_group       = MutuallyExclusiveCallbackGroup() # odometry
    ↪ subscription
self.control_group     = MutuallyExclusiveCallbackGroup() # 100 Hz control
    ↪ law
self.rollout_group     = MutuallyExclusiveCallbackGroup() # reachable-tube
    ↪ rollouts
self.estimated_group   = MutuallyExclusiveCallbackGroup() # wind EKF + LQR
    ↪ gain updates

self.control_timer = self.create_timer(0.01,
    ↪ self.control_algorithm_callback,
                                     callback_group=self.control_group)
```

Why mutually exclusive and not reentrant? A reentrant control group would let a late control tick overlap with the next one, so two threads would write `last_input` at the same time. Each job here should never overlap *with itself*; it only needs to overlap with *the other jobs*. One mutually exclusive group per job says exactly that. It also means the executor never needs more threads than there are groups, so the node uses `MultiThreadedExecutor(num_threads=5)`.

3.4 StaticSingleThreadedExecutor (rclcpp only)

A C++ executor that builds its wait set **once**, when entities are added, instead of rebuilding it on every loop iteration. Most nodes never create or destroy timers or subscriptions after start-up, so that rebuilding is pure overhead, and removing it measurably reduces CPU use and latency for C++ nodes with many entities. It is still a *single* thread: it makes dispatch cheaper but can't run two callbacks at once, so a 165 ms callback still blocks everything else.

It isn't available in rclpy. In Python, the Python-level machinery around each callback (building the wait set, generators, futures) is much more expensive than in C++, and the closest thing to a “static”/low-overhead executor is the next one.

3.5 EventsExecutor (rclcpp, and rclpy.experimental since Jazzy)

Instead of a wait set that has to be rebuilt and scanned, middleware events (a message arrived, a timer expired) are pushed into a queue, and the executor pops and dispatches them. In rclpy, the **EventsExecutor** is implemented in C++, so the per-callback overhead of the Python executors largely disappears. It is still **single-threaded**. That makes it ideal when no callback is long, which is exactly what the process backend (below) arranges.

```
from rclpy.experimental import EventsExecutor
executor = EventsExecutor()
executor.add_node(node)
executor.spin()
```

The node exposes all three with `--executor {multi,single,events}`.

4 Python-specific: the GIL

CPython has a **Global Interpreter Lock**: only one thread at a time may execute Python bytecode. Two Python threads doing pure-Python arithmetic take turns. They never run simultaneously. The interpreter makes a running thread hand over the GIL when another thread asks for it, after the *switch interval* (`sys.getswitchinterval()`, 5 ms by default).

So why can threads help at all? **C extensions may release the GIL while they work.** NumPy's large array operations, LAPACK calls, file and socket I/O, `time.sleep`, and XLA executing a compiled JAX function all release it. While the rollout thread is inside the compiled `jitted_rollout` (a 3000-step `lax.scan` running in XLA's C++ runtime), it doesn't hold the GIL, and the control thread is free to run Python.

That was measured, not assumed. With a rollout thread computing back-to-back rollouts, a control-sized JAX call in the main thread took 0.29 ms on average (1.07 ms at worst). A 100 Hz timer next to back-to-back rollouts on a `MultiThreadedExecutor` held 100.0 Hz, with a worst period of 12.6 ms. The same pair on a `SingleThreadedExecutor` ran at **6 Hz**.

The GIL still matters in three ways:

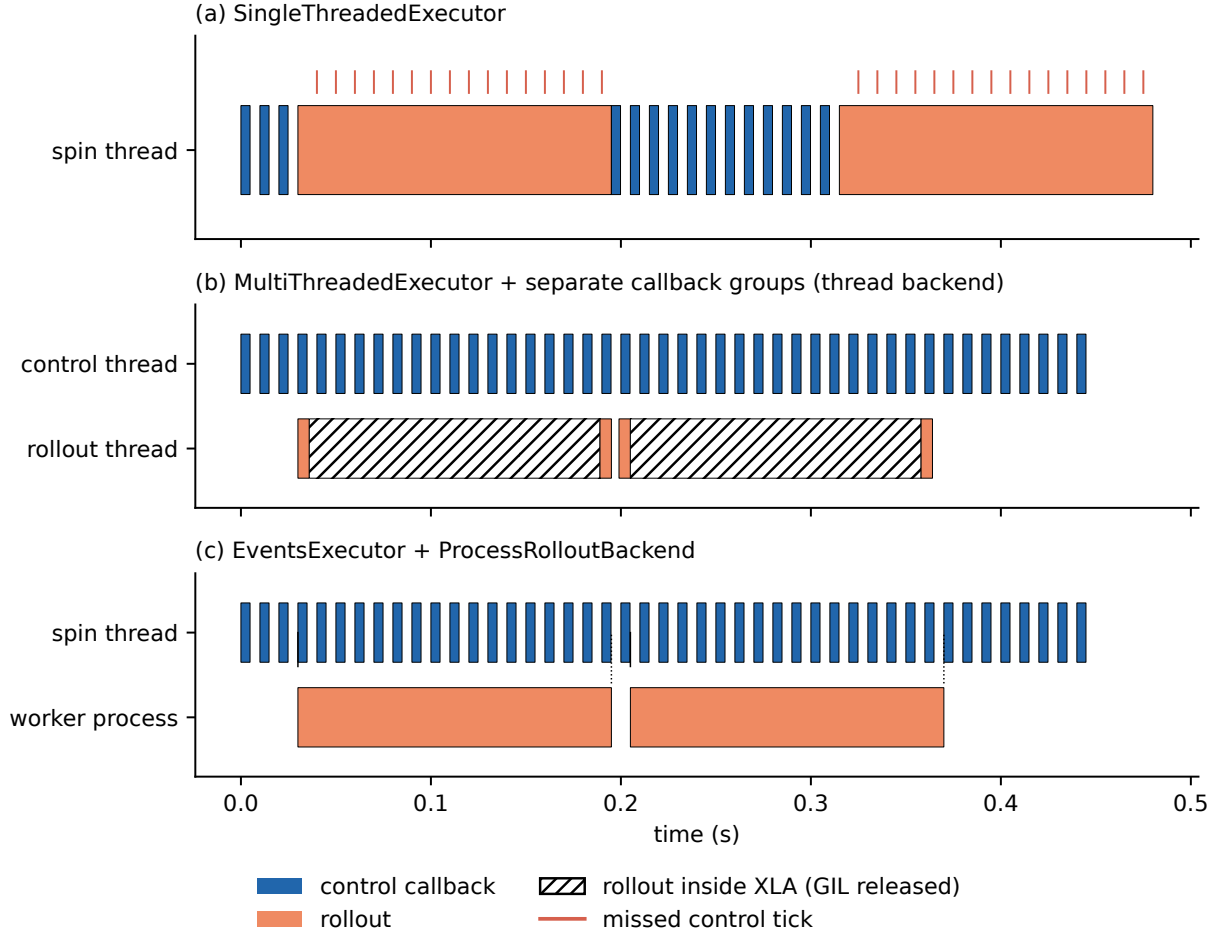


Figure 1: Schedules on this machine (control callback 1.6 ms, rollout 165 ms). (a) One thread: every control tick that falls inside a rollout is lost. (b) Separate callback groups on a MultiThreadedExecutor: the rollout thread holds the GIL only briefly at the start and end; while XLA crunches the scan, the GIL is free and the control thread runs on schedule. (c) The rollout lives in another process; the executor thread only submits requests and polls for results, which takes microseconds.

1. **Python glue around the heavy call** (building arguments, `irx.icentpert`, converting results) holds the GIL. It is short here, but it isn't free.
2. **Anything that holds the GIL for a long time freezes every thread.** The garbage collector is the worst offender (next section), and some C extensions forget to release the GIL.
3. **The executor itself is Python.** `rcipy`'s `MultiThreadedExecutor` runs its dispatch loop in Python on the spinning thread, and that thread competes for the GIL like everyone else.

4.1 Thread safety with a GIL

The GIL makes individual bytecode operations atomic. In particular, **rebinding an attribute** (`self.plan = new_plan`) is **atomic**: a reader gets either the old object or the new one, never a torn mix. The GIL does *not* make a *sequence* of operations atomic. The original odometry callback did this:

```
self.x = msg.position[0]
self.y = msg.position[1]
...
self.rta_mm_gpr_state_vector_planar = np.array([self.y, self.z, ...])
```

<- another thread can run here

A control thread reading `self.x`, then `self.vy`, could see position from one sample and velocity from the next. Two patterns fix this without adding locks to the hot path:

Immutable snapshot, published with one assignment. Build the *whole* new value, then publish it with a single rebinding. Readers take one reference at the top of the callback and use only that:

```
# writer (odometry thread)
self.state = VehicleState(x=..., y=..., vy=..., nr_state_vector=..., ...)
↪ # frozen dataclass

# reader (control thread)
s = self.state          # one atomic read; s never changes underneath us
u = controller(s.nr_state_vector, ...)
```

The same pattern carries rollout results to the controller (`RolloutPlan`) and LQR gains (`self.gains = (K_feedback, K_reference)`).

Copy-on-write for buffers. The GP data buffers are modified by the wind thread while the rollout thread may be reading them. The writer copies, modifies the copy, then swaps the reference:

```
new_obs = self.gz_wind_obs_in_y.copy()
new_obs[i, :] = (t, z, wind)
self.gz_wind_obs_in_y = new_obs      # the rollout thread sees old or new,
↪ never half-written
```

A `threading.Lock` is still the right tool when several fields must change *together* with a decision based on them; the node uses one when installing a plan. On a 100 Hz path, though, lock-free reads of immutable snapshots are simpler and can't deadlock.

5 Threads vs. processes

A **process** has its own interpreter, its own memory and its own GIL. Work in another process can't be stalled by this process's GIL, garbage collector or executor, and vice versa. The price:

- **No shared Python objects.** Inputs and results are *pickled*, sent through a pipe and unpickled. For a rollout that's about 0.4 MB per result (tube 3001×10, reference 3001×5, inputs 3000×2, all float64), which takes about a millisecond.
- **Each process initialises its own JAX runtime and JIT-compiles its own functions.** The worker starts first so its roughly 10 s of compilation overlaps with the node's own compilation.
- **Start method.** `fork` duplicates the parent process, but only the thread that called `fork`. A parent that has already started JAX/XLA thread pools and ROS 2 middleware threads can leave the child with locks held by threads that no longer exist, a classic deadlock. (Python 3.12 warns about forking a multithreaded process for this reason.) The worker therefore uses **spawn**: a fresh interpreter that imports the module and runs only the worker function. One consequence: the script that starts a spawned process must guard its entry point with `if __name__ == '__main__':`, or the child re-runs the parent's top-level code.

The node's `ProcessRolloutBackend` keeps the child on one duplex `multiprocessing.Pipe`:

```
backend.submit(request)      # pickles a RolloutRequest into the pipe;
↪ returns immediately
result = backend.poll()      # conn.poll(0): None, or the unpickled
↪ RolloutResult
```

Because neither call blocks, **even a single-threaded executor keeps its control loop at 100 Hz** while a rollout is computing: the rollout timer just checks “is it done yet?” every 10 ms. `--rollout-cpus 2,3` additionally pins the worker to specific cores with `os.sched_setaffinity`, so it never competes with the control thread for a core.

When to use which:

	thread backend	process backend
needs a multithreaded executor	yes	no
affected by this process's GIL / GC	yes (only the Python glue)	no
start-up cost	none	second JAX runtime + JIT (~10 s, in parallel)
per-rollout overhead	none	pickle + pipe (~1 ms)
memory	shared	a second copy of JAX and the model

6 Stop-the-world pauses that aren't your code

6.1 Garbage collection

CPython frees most objects immediately by reference counting. Objects in reference *cycles* are found by a separate **cyclic garbage collector**. It sorts container objects (lists, dicts, class instances, ...) into three generations and periodically scans them. A scan of the oldest generation, a **full collection**, walks *every* tracked object, **holding the GIL the whole time**. No other Python thread runs during it, whatever the executor.

The node appends about 50 small Python lists per control tick to its log buffers (the logged reachable-tube rows), so the set of tracked objects grows all flight. Full collections are triggered when the long-lived set has grown by 25 %, so they get rarer but *longer*. Measured in flight: **4–5 full collections of about 300 ms each**, which is 30 missed control ticks each time, at unpredictable moments (Figure 2).

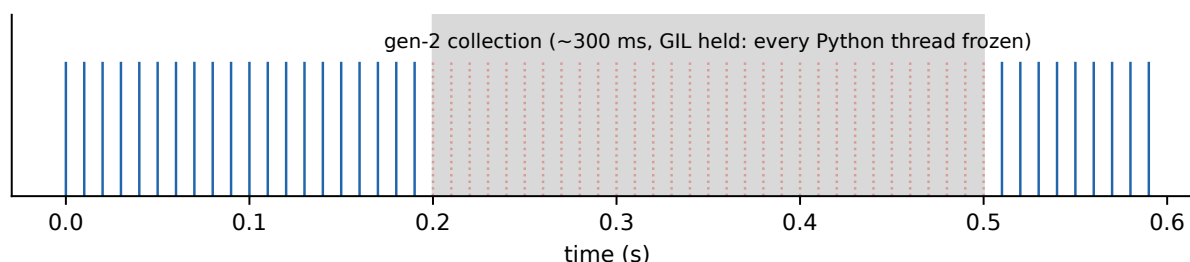


Figure 2: A 300 ms full garbage collection holds the GIL: the control thread can't run, regardless of executor or callback groups.

Two standard tools for real-time Python:

- **`gc.freeze()`** moves every object that exists right now into a permanent generation that the collector never scans again. Called after initialisation, it removes the JAX/XLA caches, compiled functions and ROS entities (about 270 000 objects here) from every future scan.
- **Raising the third threshold** (`gc.set_threshold(700, 10, 10**9)`) stops automatic full collections. Young-generation collections still run (1–3 ms, they only look at new objects), and the node calls `gc.collect()` at shutdown. For a flight of a few minutes, the extra memory from uncollected cycles is negligible.

`gc.freeze()` alone did *not* fix this node. The slow scans were caused by objects created *during* flight, which is why the heap report at shutdown matters: it showed 23 449 of 24 080 tracked objects were `lists`, one per logged tube row.

6.2 Just-in-time compilation in flight

`jax.jit` compiles a function the first time it sees a new *signature*: the argument shapes and dtypes, plus a few properties such as whether an array is already placed (“committed”) on a device. The original node warmed up its jitted functions with *test* inputs, but in flight the arguments arrived with a different signature, so XLA recompiled mid-air:

t (s)	compiled in flight	time
15.34	NR_tracker_original	322 ms
15.37	dynamic_slice, concatenate	43 ms
15.50	dynamics (wind thread)	60 ms

The trigger was the second RTA tick: `self.last_input` switches from the array created at start-up to the controller’s own output (`jnp.hstack(...)`), which JAX treats as a different signature. A first fix was to warm up **with the values the loop actually feeds back**. The robust fix (document 03) is **ahead-of-time compilation**: `jax.jit(f).lower(shapes).compile()` produces an executable for one signature that can never recompile, and every in-flight argument is converted to a float64 NumPy array first. To find such stalls, run with `JAX_LOG_COMPILE=1` and look for compilation messages after take-off.

6.3 Printing

`print()` at 100 Hz from several callbacks, sometimes of whole messages or JAX arrays (which forces a device-to-host copy and formatting), holds the GIL and can block on the terminal. The original control tick spent about 9 ms of its 10 ms budget in printing and JAX indexing. Hot-path printing is now behind `--verbose`. A cheap 1 Hz status line remains.

7 Checklist for a Python ROS 2 control node

1. Put every periodic job in **its own `MutuallyExclusiveCallbackGroup`**, and spin a **`MultiThreadedExecutor`** with one thread per group.
2. Keep the control callback **short and GIL-light**: NumPy/JAX calls on small arrays, no printing, no per-tick allocation of Python containers you keep.
3. Move anything slow (optimisation, rollouts, learning) to **another group**, or to **another process** when it’s mostly Python or when isolation matters.
4. Share data between threads through **immutable snapshots, swapped with one assignment**.
5. **Warm up every jitted function with in-flight argument types**, and check with `JAX_LOG_COMPILE=1`.
6. **Control the garbage collector** (`gc.freeze()` after init, no automatic full collections in flight).
7. **Measure**: log the achieved period of the control loop, not just its compute time. A loop that “takes 1.6 ms” can still run at 12 Hz if something else holds the thread.